

2026-05-19-recovery-batch-verification

Round-193 §11.4 — Recovery-Batch Verification Report

Date: 2026-05-19 **Round:** 193 (verification audit, no production code changes) **Scope:** Re-verification of 10 packages whose CONST-046 i18n recovery content was landed via Phase-4 recovery batches (b7f8672, 5c94696) without per-round centralized test execution. **Cascade authority:** CONST-035 / CONST-049 / Article XI §11.9

Verbatim Operator Mandate (per CONST-049 §11.4.17)

Verbatim 2026-05-19 operator mandate: “all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”

Methodology

For each of 10 packages landed via recovery batches in rounds 157, 161, 163, 165, 166-RE, 167, 168, 173, 175, 178:

1. `go build ./internal/<pkg>/... (from helix_code/)`
2. `go test -count=1 ./internal/<pkg>/... (from helix_code/)`
3. `go run scripts/audit_const046/main.go --roots helix_code/internal/<pkg> --json (from repo root)`

PASS = clean build AND all tests green. FAIL = build error OR test failure (CONST-046 audit violations recorded separately as advisory; pre-existing migration debt, not blocker for this audit).

Toolchain: `go test -count=1` defeats test cache so every PASS is a fresh runtime observation per Article XI §11.9.

Summary

#	Package	Round	Build	Tests	CONST-046 audit	Overall
1	focus	157	PASS	PASS	1 violation	PASS
2	llm	161	PASS	FAIL	many violations	FAIL
3	logo	163	PASS	FAIL	1 violation	FAIL

#	Package	Round	Build	Tests	CONST-046 audit	Overall
4	memory	165	PASS	PASS	many violations	PASS
5	notification	167	PASS	FAIL	many violations	FAIL
6	performance	168	FAIL	FAIL	many violations	FAIL
7	monitoring	166-RE	PASS	PASS	0 violations	PASS
8	redis	173	PASS	PASS	0 violations	PASS
9	rules	175	PASS	PASS	0 violations	PASS
10	session	178	PASS	PASS	1 violation	PASS

Tally: 6 PASS / 4 FAIL (40 % recovery-batch failure rate — confirms hypothesis that uncentralized recovery batches under-verify).

Failure Details + Proposed Remediation

FAIL-1: `internal/llm` — **wizard test asserts substring, message ID is bare token**

Symptom:

```
--- FAIL: TestValidateWizardForm_AnthropicRequiresAPIKey (0.00s)
    wizard_test.go:55: error "internal_llm_wizard_anthropic_apikey_require
```

Root cause: Round-161 i18n migration replaced the human-readable error string with a CONST-046 message ID (`internal_llm_wizard_anthropic_apikey_required`). The test was written against the old literal substring (`api_key`) and was not updated in the same commit. Under `NoopTranslator{}` the message ID echoes literally, so the substring check fails.

Proposed remediation (separate scope, NOT executed in this round): - Option A (preferred): tighten the test to assert the message-ID literal (`require.Contains(t, err.Error(), "internal_llm_wizard_anthropic_apikey_required")`). Preserves anti-bluff guarantee that the wizard surfaces the correct error key. - Option B: install a test-local `Translator` that returns a human-readable translation containing `api_key`, then keep the substring assertion. Demonstrates full i18n round-trip but requires test-local YAML fixture.

FAIL-2: `internal/logo` — **image-decode error wrapping wraps i18n key, not English phrase**

Symptom:

```
--- FAIL: TestGenerateASCIIArt/generate_ASCII_from_invalid_image
    processor_test.go:219: "internal_logo_decode_source_failed" does not c
--- FAIL: TestGenerateASCIIArt/generate_ASCII_from_missing_file
```

Root cause: Same class as FAIL-1. Round-163 migration replaced the error literal `"failed to decode source"` with message ID `internal_logo_decode_source_failed`. Test still expects the English substring.

Proposed remediation: Same as FAIL-1 — update test to assert the message ID OR wire a test Translator with English bundle.

FAIL-3: internal/notification — 8 event-handler tests assert English title literals

Symptom: 8 sibling failures including

TestEventNotificationHandler_HandleEvent_TaskCompleted:

expected: "Task Completed"

actual : "internal_notification_title_task_completed"

Root cause: Round-167 migration replaced all 8 notification title literals with CONST-046 message IDs. Tests still assert English titles. Under NoopTranslator{} the IDs echo literally.

Proposed remediation: Update 8 tests to assert message-ID literals (same pattern as FAIL-1/FAIL-2). Alternatively, install a test bundle that decodes the IDs to English. Affected tests: _TaskCompleted, _TaskFailed, _WorkflowCompleted, _WorkflowFailed, _WorkerDisconnected, _SystemError, _SystemStartup, _EndToEnd.

FAIL-4: internal/performance — build broken: undefined stdctx, unused context

Symptom (compile failure, blocks both build and test):

```
internal/performance/translator.go:14:2: "context" imported and not used
internal/performance/translator.go:49:13: undefined: stdctx
```

Root cause: Round-168 migration introduced translator.go referencing stdctx.Context (per the pattern used by sibling packages that aliased "context" as stdctx because they also imported a domain-local context package). internal/performance does NOT have a domain-local context collision, so the import is written as import "context" not import stdctx "context". The tr(...) helper still uses stdctx.Context. Net: import unused, identifier undefined.

Proposed remediation: Edit helix_code/internal/performance/translator.go line 49:

```
// Change:
func tr(ctx stdctx.Context, msgID string, data map[string]any)
    string {
// To:
func tr(ctx context.Context, msgID string, data map[string]any)
    string {
```

Removes the unused-import error and the undefined-identifier error in one keystroke. Then re-run build + tests. Pre-existing CONST-046 violations (many) remain as separate migration debt — out of scope for this audit.

CONST-046 audit gate observations (advisory)

Packages with zero CONST-046 violations (clean migrations): `monitoring`, `redis`, `rules`. Packages with single residual violation: `focus` (1), `logo` (1), `session` (1). Packages with batched residuals carried forward: `llm`, `memory`, `notification`, `performance` (each with multiple hardcoded-string hits — pre-existing migration debt tracked separately by Phase-4 sweep).

The audit gate result is advisory for this round; gating closure of these violations is the Phase-4 round-by-round migration's responsibility, not this verification audit's.

Conclusion

40 % of the 10 recovery-batch packages failed verification (4 of 10). Three failures (`llm`, `logo`, `notification`) are test-assertion drift from the `il8n` migration — tests not updated in the same commit as the production code change. One failure (`performance`) is a compile-blocking build error from copy-paste of the sibling-package translator template without adapting the import alias.

All four failures are §11.4 PASS-bluffs at the recovery-batch layer: the recovery batches reported successful content capture but the resulting trees do not build/test green. This confirms the operator's 2026-05-19 mandate: tests-pass-as-summary-line without runtime evidence is no longer acceptable as a closure signal — even for stalled-agent recovery flows.

Remediation per failure is scoped above. Fixes will be executed in a follow-up round (separate scope per the round-193 charter — this round documents, does not fix).

Provenance correction

The initial filesystem-write of this file at round-193 was captured by a parallel session's commit `c41bc2f` (round-190 HXA-003 venice fix) due to a race in pre-commit staging. The audit content is unchanged; this commit re-anchors the round-193 audit to its proper subject line per the round-193 charter.